Practica Integradora Final

Objetivo general

Integrar herramientas y metodologías avanzadas del ciclo de vida del software mediante la implementación de una aplicación web que contemple gestión ágil, control de versiones, contenedores, bases de datos optimizadas y replicadas, integración y despliegue continuos, monitoreo, gestión de incidencias, documentación, automatización con n8n y ciberseguridad.

Desarrollo cronológico del proyecto

1. Selección y diagnóstico de la aplicación

Seleccionar una aplicación web existente y conocer su arquitectura antes de modificarla.

Actividades

1. Seleccionar una aplicación académica, institucional o desarrollada previamente.
2. Identificar sus funcionalidades principales.
3. Documentar el stack tecnológico utilizado.

Se deberá identificar (tomar en cuenta los últimos eventos de ciberseguridad):

- **Backend:** Node.js, Spring Boot, Django, PHP u otra tecnología.
- **Frontend:** Angular, React, Vue u otra tecnología.
- **Base de datos:** PostgreSQL o MySQL/MariaDB.
- **Herramientas adicionales:** bibliotecas, API, servicios externos y dependencias.
- **Infraestructura actual:** sistema operativo, servidor, puertos y forma de ejecución.

4. Determinar qué componentes deberán contenerizarse.
5. Elaborar un diagrama inicial de arquitectura.

Si la aplicación no dispone de un repositorio remoto, se deberá inicializar Git y publicarla en GitHub o GitLab.

Entregables

- Descripción de la aplicación.
- Inventario del stack tecnológico.
- Diagrama inicial de arquitectura.
- Lista de funcionalidades.
- Diagnóstico técnico y plan de mejoras.

2. Gestión del proyecto mediante Scrum

Organizar el trabajo y establecer un mecanismo verificable para controlar el avance.

Actividades

1. Crear el proyecto en Jira o una herramienta equivalente.

2. Definir el Product Backlog con las funcionalidades y tareas técnicas.
3. Establecer prioridades, responsables y criterios de aceptación.
4. Dividir el trabajo en sprints.
5. Registrar las tareas y sus estados.
6. Realizar reuniones breves de seguimiento.
7. Documentar impedimentos, cambios de alcance y decisiones.

El backlog deberá incluir no solamente funcionalidades de la aplicación, sino también infraestructura, pruebas, seguridad, documentación y despliegue.

Entregables

- Proyecto en Jira o herramienta equivalente.
- Product Backlog.
- Plan de sprints.
- Responsables y criterios de aceptación.
- Evidencias de seguimiento del proyecto.

3. Control de versiones y calidad del código

Establecer un flujo de trabajo colaborativo, trazable y seguro.

Actividades

1. Crear o reorganizar el repositorio.
2. Definir una estrategia de ramas, por ejemplo:
    - main: versión estable.
    - develop: integración del desarrollo.
    - feature/*: nuevas funcionalidades.
    - fix/*: correcciones.
3. Realizar commits descriptivos y vinculados con las tareas.
4. Implementar Merge Requests o Pull Requests.
5. Revisar el código antes de integrar cambios.
6. Resolver conflictos y registrar correcciones.
7. Proteger la rama principal y restringir las modificaciones directas.

Se deberá evitar subir contraseñas, tokens, claves privadas, archivos .env con secretos o copias de bases de datos con información sensible.

Entregables

- Repositorio remoto público, sin secretos ni datos personales reales.
- Historial de commits.

- Ramas de trabajo.
- Solicitudes de integración y revisiones de código.
- Archivo README inicial.

4. Contenerización de la base de datos y los servicios auxiliares

Preparar un entorno reproducible y aislado mediante Docker.

Actividades

1. Crear un contenedor para PostgreSQL o MySQL/MariaDB.
2. Definir volúmenes persistentes.
3. Configurar una red interna de Docker.
4. Incorporar un cliente de administración dockerizado, como pgAdmin o phpMyAdmin.
5. Migrar la estructura y los datos existentes.
6. Configurar variables de entorno y credenciales.
7. Incorporar un servicio de backups.
8. Comprobar que los datos persisten al recrear los contenedores.

Se deberá utilizar Docker Compose para describir los servicios, redes, volúmenes y dependencias.

Las credenciales no deberán quedar escritas directamente en el repositorio público.

Dataset de prueba

Se deberá preparar un conjunto de datos de prueba de al menos 6.000.000 registros por tabla de datos que forme parte del conjunto de prueba de la base de datos.

Los datos podrán generarse de forma sintética, respetando las relaciones entre tablas.

Se deberá documentar:

- Cantidad de registros por tabla.
- Tamaño de la base de datos.
- Método de generación o importación.
- Tiempo de carga.
- Restricciones de integridad.
- Tiempo y resultado de las copias de seguridad.

Herramientas de análisis

Se utilizarán herramientas apropiadas para el motor seleccionado, como MySQLTuner, Percona Toolkit, pgBadger u otras equivalentes.

Entregables

- Dockerfile y compose.yaml o docker-compose.yml.
- Esquema de la base de datos.

- Script o procedimiento de carga.
- Evidencia de volumen de datos.
- Cliente de administración funcionando.
- Backup y evidencia de restauración.

5. Optimización y análisis de la base de datos

Identificar y corregir problemas de rendimiento a partir de mediciones.

Actividades

1. Identificar consultas frecuentes o costosas.
2. Analizar las consultas con EXPLAIN y, cuando esté disponible, EXPLAIN ANALYZE.
3. Examinar índices, cardinalidad, selectividad y estadísticas.
4. Detectar recorridos completos de tablas y uniones costosas.
5. Evaluar índices simples, compuestos y de cobertura cuando correspondan.
6. Revisar consultas no SARGables.
7. Optimizar al menos 8 consultas relevantes.
8. Medir y comparar los resultados antes y después.
9. Analizar parámetros de configuración del motor.
10. Documentar las decisiones y sus efectos.

No se aceptará la creación indiscriminada de índices sin justificar su utilidad y su impacto en inserciones, actualizaciones y almacenamiento.

Entregables

- Consultas originales y optimizadas.
- Planes de ejecución iniciales y posteriores.
- Índices agregados y justificación.
- Mediciones de rendimiento.
- Informe de optimización.

6. Clúster y replicación de la base de datos

Demostrar la distribución de datos entre nodos y evaluar la continuidad del servicio.

Actividades

1. Implementar una arquitectura de replicación compatible con el motor seleccionado.
2. Configurar al menos un nodo principal y un nodo réplica.
3. Documentar los roles de cada nodo.
4. Verificar que las operaciones se repliquen correctamente.
5. Configurar la aplicación para conectarse a la arquitectura implementada.

6. Evaluar consistencia, retraso de replicación y recuperación.
7. Simular la indisponibilidad de un nodo en un entorno controlado.
8. Documentar las limitaciones y el procedimiento de recuperación.

La replicación por sí sola no garantiza alta disponibilidad automática. Si el equipo afirma contar con failover automático, deberá demostrar cómo se detecta la falla, cómo se promueve otro nodo y cómo se evita la pérdida o corrupción de datos.

Entregables

- Diagrama de replicación.
- Configuraciones de los nodos.
- Evidencias de replicación.
- Prueba de fallo y recuperación.
- Informe sobre consistencia y limitaciones.

7. Contenerización de la aplicación web

Ejecutar todos los componentes de la aplicación en servicios aislados y reproducibles.

Actividades

1. Crear un contenedor para el backend.
2. Crear un contenedor para el frontend.
3. Conectar ambos servicios a la red correspondiente.
4. Configurar la conexión con la base de datos.
5. Separar las configuraciones de desarrollo y despliegue.
6. Verificar la persistencia de datos.
7. Comprobar el funcionamiento de la aplicación desde el navegador.

Entregables

- Contenedores funcionales.
- Configuración Docker Compose.
- Evidencias de comunicación entre frontend, backend y base de datos.
- Capturas de la aplicación en funcionamiento.

8. Integración continua - Despliegue continuos (CI/CD)

Automatizar construcción y publicación de nuevas versiones.

Actividades

1. Configurar GitHub Actions, GitLab CI/CD o Jenkins.
2. Ejecutar verificaciones automáticas del código.

3. Construir las imágenes Docker.
4. Publicar las imágenes en un registro cuando corresponda.
5. Configurar el despliegue automático en el entorno elegido.
6. Proteger los secretos y credenciales del pipeline.
7. Ejecutar un commit de prueba.
8. Verificar que el cambio llegue al entorno publicado.
9. Documentar fallos y mecanismos de reversión.

El pipeline deberá evitar que una modificación defectuosa se publique sin las validaciones definidas por el equipo.

Entregables

- Archivo de configuración del pipeline.
- Ejecución exitosa de CI/CD.
- Evidencia del commit de prueba.
- Evidencia de despliegue.
- Procedimiento de recuperación o rollback.

9. Publicación y acceso desde Internet

Permitir que la aplicación pueda utilizarse desde una red externa, con controles de seguridad adecuados.

Servidor local con túnel seguro

Ejecutar la aplicación en una máquina local o máquina virtual y habilitar el acceso externo mediante una solución segura.

Se podrán utilizar, según la arquitectura, WireGuard, Tailscale, Cloudflare Tunnel u otra herramienta equivalente.

Se deberá:

1. Configurar el servidor local.
2. Implementar el túnel o mecanismo de publicación.
3. Definir quién puede acceder a la aplicación.
4. Configurar HTTPS y autenticación cuando corresponda.
5. Evitar exponer directamente puertos de administración o de base de datos.
6. Verificar el acceso desde un dispositivo situado fuera de la red local.

Requisitos comunes

- La aplicación deberá estar accesible desde Internet durante la demostración final.
- Se deberán exponer solamente los servicios necesarios.
- No se deberá publicar directamente la base de datos.

- Las interfaces administrativas deberán estar restringidas.
- Se deberá documentar la arquitectura de acceso.
- Se deberá medir la disponibilidad y registrar los errores relevantes.

Entregables

- URL o mecanismo de acceso documentado.
- Diagrama de red.
- Configuración de publicación.
- Evidencia de acceso externo.
- Configuración del firewall y HTTPS.
- Análisis de riesgos y limitaciones de la alternativa elegida.

10. Monitoreo y visualización con Prometheus y Grafana

Observar el estado y el rendimiento de la aplicación y de su infraestructura.

Actividades

1. Desplegar Prometheus.
2. Desplegar Grafana.
3. Incorporar exportadores adecuados para el sistema operativo, Docker y la base de datos.
4. Configurar la recolección de métricas.
5. Construir un dashboard.
6. Definir umbrales y alertas.
7. Generar una situación de prueba para comprobar las alertas.

El dashboard deberá incluir, como mínimo:

- CPU y memoria.
- Uso de disco.
- Red.
- Estado y reinicios de contenedores.
- Disponibilidad de la aplicación.
- Métricas de la base de datos.
- Conexiones activas y consultas lentas, cuando el motor y el exportador lo permitan.
- Estado de replicación cuando esté disponible.

No se deberá afirmar que se monitorean consultas SQL individuales si únicamente se están observando métricas generales de la base de datos.

Entregables

- Configuración de Prometheus.

- Dashboard de Grafana.
- Evidencia de métricas reales.
- Alertas configuradas.
- Prueba de detección de una condición anormal.

11. Gestión de incidencias con Mantis o GLPI

Registrar, priorizar y resolver errores durante el desarrollo y las pruebas.

Actividades

1. Instalar o desplegar Mantis Bug Tracker o GLPI.
2. Definir categorías, prioridades y responsables.
3. Registrar incidencias detectadas durante el proyecto.
4. Documentar pasos para reproducir cada problema.
5. Vincular incidencias con tareas y cambios de código.
6. Registrar la solución aplicada.
7. Verificar que la corrección funcione.
8. Cerrar la incidencia con evidencia.

Se deberán registrar al menos 10 incidencias reales o reproducibles del entorno de prueba, incluyendo al menos una relacionada con rendimiento y otra con seguridad.

Entregables

- Herramienta funcionando.
- Registro de incidencias.
- Evidencias de asignación y seguimiento.
- Historial de resolución.

12. Gestión documental con Nextcloud

Centralizar la documentación técnica y facilitar el trabajo colaborativo.

Actividades

1. Desplegar Nextcloud o utilizar una instancia autorizada.
2. Crear una estructura de carpetas.
3. Definir permisos de acceso.
4. Almacenar la documentación del proyecto.
5. Centralizar informes de pruebas, configuraciones y reportes.
6. Registrar las versiones relevantes de los entregables.
7. Evitar almacenar secretos sin protección.

8. Verificar el acceso únicamente a los usuarios autorizados.

Entregables

- Estructura documental.
- Matriz de permisos.
- Informes y evidencias centralizados.

13. Automatización de procesos con n8n

Integrar servicios y automatizar tareas operativas mediante flujos de trabajo.

Actividades

1. Desplegar n8n en Docker.
2. Configurar persistencia y protección de credenciales.
3. Integrar n8n con al menos tres componentes del proyecto.
4. Crear como mínimo tres flujos funcionales.
5. Incorporar validación de datos y manejo de errores en el flujo.
6. Registrar las ejecuciones y sus resultados.
7. Documentar los disparadores, condiciones y acciones.
8. Probar el comportamiento cuando una tarea falla en el flujo.

Flujos sugeridos

Flujo 1. Alertas de infraestructura

Prometheus detecta una anomalía → n8n recibe el evento → registra una incidencia en Mantis/GLPI → notifica al responsable.

Flujo 2. Verificación de backups

Se ejecuta una copia de seguridad → se verifica su resultado → se registra el estado → se notifica cualquier error. La restauración deberá comprobarse mediante una prueba controlada.

Flujo 3. Informe de rendimiento asistido por IA

Se recopilan métricas → un componente Python o un modelo de IA analiza anomalías → n8n organiza el resultado → el informe se almacena en Nextcloud y se notifica al equipo.

Se podrá implementar un cuarto flujo para clasificar eventos de seguridad y registrar incidentes.

Requisitos de seguridad

- Proteger la interfaz de administración.
- Autenticar los webhooks.
- Evitar secretos escritos directamente en los flujos.
- Restringir los permisos de las credenciales.
- Validar los datos recibidos.
- Controlar las acciones automáticas que puedan interrumpir servicios.

- Solicitar aprobación humana antes de ejecutar acciones críticas.

Entregables

- Configuración de n8n.
- Diagramas de los flujos.
- Evidencias de las ejecuciones.
- Registro de errores.
- Pruebas de integración.
- Análisis de riesgos de las automatizaciones.

14. Seguridad informática y amenazas actuales

Identificar, prevenir, detectar y responder ante amenazas que puedan afectar la aplicación, las bases de datos, los contenedores, la infraestructura y los procesos automatizados.

El análisis deberá considerar amenazas actuales como ransomware, DDoS, robo de credenciales, vulnerabilidades de dependencias, ataques a la cadena de suministro de software, inyección SQL, fallas de autorización, abuso de API y webhooks, y riesgos relacionados con sistemas de IA.

- Seguridad de la aplicación
- Seguridad de Docker e infraestructura
- Seguridad de CI/CD y cadena de suministro
- Pruebas controladas
- Respuesta ante incidentes

Entregables

- Matriz de amenazas.
- Informe de vulnerabilidades.
- Evidencias de escaneo y corrección.
- Resultados de pruebas controladas.
- Configuración de controles de seguridad.
- Procedimiento de respuesta y recuperación.
- Evidencias del flujo de incidentes integrado con n8n.

15. Documentación final y defensa oral

Presentar una solución reproducible y demostrar la comprensión individual de las decisiones técnicas.

Documentación técnica obligatoria

1. Descripción de la aplicación.

2. Arquitectura general.
3. Inventario tecnológico.
4. Manual de instalación.
5. Configuración Docker.
6. Procedimiento de migración y carga de datos.
7. Informe de optimización SQL.
8. Configuración y pruebas de replicación.
9. Configuración CI/CD.
10. Procedimiento de publicación en Internet.
11. Manual de monitoreo.
12. Gestión de incidencias.
13. Flujos de n8n.
14. Informe de seguridad.
15. Procedimiento de backup y restauración.
16. Limitaciones conocidas y mejoras futuras.

Presentación

Cada equipo dispondrá de aproximadamente 20 minutos.

La calificación mínima de aprobación será 7 (siete).

La nota individual considerará la calidad de los aportes, las evidencias, la comprensión de la arquitectura, las respuestas a preguntas técnicas y la capacidad para explicar decisiones y resolver problemas.

No será suficiente mostrar capturas de pantalla o afirmar que una herramienta funciona. Se deberá demostrar su funcionamiento y justificar las configuraciones realizadas.cias resueltas, pero las decisiones y pruebas deberán quedar documentadas.

Condiciones finales

- El plazo de definición de grupos es hasta el 30 de noviembre de 2026. (5 integrantes)
- La presentación final se realizará durante la semana del 16 de noviembre de 2026.
- Cada equipo deberá mantener actualizado su repositorio y el tablero de trabajo.
- La aplicación deberá estar accesible desde Internet al momento de la demostración.
- La calificación será individual, aun cuando el proyecto se desarrolle en equipo. Para aprobar, cada estudiante deberá obtener una nota igual o superior a 7 (siete).
- La cátedra podrá tomar un examen individual, teórico y/o práctico, cuando lo considere necesario, a fin de verificar la participación efectiva de cada estudiante

en el desarrollo del proyecto, su comprensión de los temas abordados y su capacidad para fundamentar las decisiones técnicas adoptadas.

- Cada integrante deberá acreditar el dominio de los componentes implementados, las herramientas utilizadas y los procedimientos de configuración, integración, despliegue, optimización y seguridad aplicados en el proyecto.